You said:

Linear regression Machine Learning model Development from scratch

ChatGPT said:

Absolutely. Let's build a **Linear Regression Machine Learning model from scratch**, without using `sklearn` for the actual model.

1. What is Linear Regression?

Linear Regression is a **supervised machine learning algorithm** used to predict a continuous value.

For one feature:

$$\hat{y} = b_0 + b_1 x$$

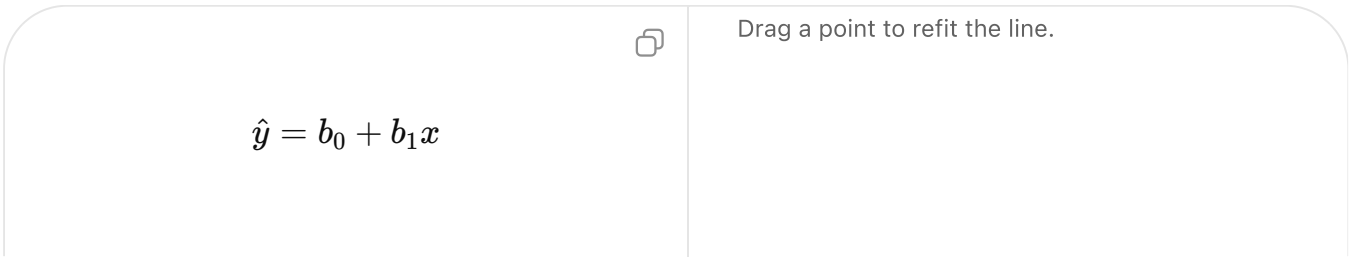
Where:

- x = input feature
- $\hat{y}$ = predicted value
- b_0 = intercept
- b_1 = slope/weight

For example, predicting salary from years of experience:

Experience	Salary
1	35
2	40
3	50
4	60
5	65

The model tries to find the best line through these points.



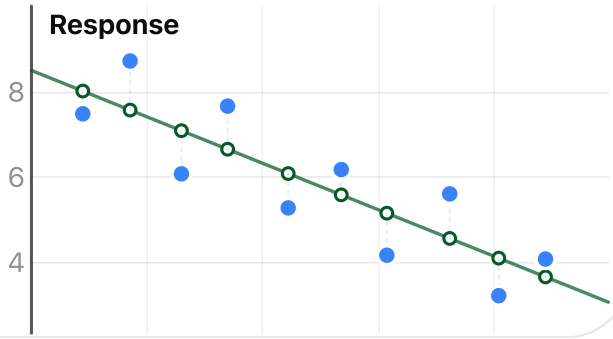
$$\hat{y} = 8.49 - 0.54x$$

Estimated intercept · R-squared = 0.72

Fit Intercept Through origin

Focus Data Line Predictions Residuals

Generate new data



Give feedback

---

2. Complete ML workflow

A typical Linear Regression project follows:

```
Dataset
   ↓
Data Cleaning
   ↓
Feature Selection
   ↓
Train/Test Split
   ↓
Initialize weights
   ↓
Prediction
   ↓
Calculate Error
   ↓
Calculate Gradient
   ↓
Update weights
   ↓
Repeat
   ↓
Final Model
   ↓
Evaluate
   ↓
Predict new data
```

---

3. Create a dataset

Let's first create a simple dataset.

```
import numpy as np

X = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([35, 40, 50, 60, 65], dtype=float)
```

Here:

```
X = Years of Experience
y = Salary
```

---

4. Linear Regression equation

Our model is:

```
y_pred = w * X + b
```

Where:

```
w = weight/slope
b = bias/intercept
```

Initially, we don't know `w` and `b`.

So initialize:

```
w = 0
b = 0
```

---

5. Prediction function

```
def predict(X, w, b):
    return w * X + b
```

Example:

```
w = 2
b = 5

prediction = predict(X, w, b)
```

```
print(prediction)
```

Output:

```
[ 7.  9. 11. 13. 15.]
```

Obviously, these predictions aren't good yet.

We need to **train the model**.

---

6. Cost Function

We need a way to measure how wrong our predictions are.

The most common cost function is **Mean Squared Error (MSE)**:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Python:

```
def cost_function(X, y, w, b):

    predictions = predict(X, w, b)

    error = y - predictions

    cost = np.mean(error ** 2)

    return cost
```

Test it:

```
w = 0
b = 0

cost = cost_function(X, y, w, b)

print(cost)
```

---

7. Gradient Descent

Now comes the important part.

We want to find the best values of:

w
b

Gradient Descent repeatedly changes them to reduce the error.

The gradients are:

$$\frac{\partial J}{\partial w} = -\frac{2}{n}\sum X(y - \hat{y})$$

$$\frac{\partial J}{\partial b} = -\frac{2}{n}\sum (y - \hat{y})$$

Then:

$$w = w - \alpha \frac{\partial J}{\partial w}$$

$$b = b - \alpha \frac{\partial J}{\partial b}$$

where:

α = learning rate

---

8. Implement Gradient Descent

```
def train(X, y, learning_rate=0.01, epochs=1000):

    w = 0.0
    b = 0.0

    n = len(X)

    for i in range(epochs):

        # Prediction
        y_pred = w * X + b
```

```
        # Error
        error = y_pred - y

        # Gradients
        dw = (2 / n) * np.sum(X * error)
        db = (2 / n) * np.sum(error)

        # Update parameters
        w = w - learning_rate * dw
        b = b - learning_rate * db

    return w, b
```

Train the model:

```
w, b = train(X, y)

print("Weight:", w)
print("Bias:", b)
```

The model has now learned the relationship between:

```
Experience → Salary
```

---

9. Make predictions

```
y_pred = predict(X, w, b)

print(y_pred)
```

You can predict a new candidate's salary:

```
experience = 6

salary = predict(experience, w, b)

print("Predicted salary:", salary)
```

---

10. Calculate MSE

```
mse = np.mean((y - y_pred) ** 2)
```

```
print("MSE:", mse)
```

Lower MSE generally means the predictions are closer to the actual values.

---

11. Calculate R² Score

Another important metric is **R²**.

$$R^2 = 1 - \frac{\sum(y - \hat{y})^2}{\sum(y - \bar{y})^2}$$

Implement it ourselves:

```
def r2_score(y, y_pred):

    ss_res = np.sum((y - y_pred) ** 2)

    ss_total = np.sum((y - np.mean(y)) ** 2)

    r2 = 1 - (ss_res / ss_total)

    return r2
```

Use:

```
r2 = r2_score(y, y_pred)

print("R² Score:", r2)
```

Interpretation:

```
R² = 1       → perfect prediction
R² ≈ 0.9     → very good
R² ≈ 0.5     → moderate
R² ≈ 0       → poor
R² < 0       → worse than mean prediction
```

---

12. Complete Linear Regression from scratch

Here is the complete implementation in one program:

```
import numpy as np
```

```
class LinearRegression:

    def __init__(self, learning_rate=0.01, epochs=1000):

        self.learning_rate = learning_rate
        self.epochs = epochs

        self.w = 0.0
        self.b = 0.0


    def fit(self, X, y):

        n = len(X)

        for i in range(self.epochs):

            # Prediction
            y_pred = self.w * X + self.b

            # Error
            error = y_pred - y

            # Gradients
            dw = (2 / n) * np.sum(X * error)

            db = (2 / n) * np.sum(error)

            # Update weights
            self.w = self.w - self.learning_rate * dw

            self.b = self.b - self.learning_rate * db


    def predict(self, X):

        return self.w * X + self.b


    def mse(self, y, y_pred):

        return np.mean((y - y_pred) ** 2)
```

```
    def r2_score(self, y, y_pred):
        ss_res = np.sum((y - y_pred) ** 2)
        ss_total = np.sum((y - np.mean(y)) ** 2)
        return 1 - (ss_res / ss_total)

# Dataset
X = np.array([1, 2, 3, 4, 5], dtype=float)
y = np.array([35, 40, 50, 60, 65], dtype=float)

# Create model
model = LinearRegression(
    learning_rate=0.01,
    epochs=1000
)

# Train
model.fit(X, y)

# Prediction
predictions = model.predict(X)

# Results
print("Weight:", model.w)
print("Bias:", model.b)
print("Predictions:", predictions)
print("MSE:", model.mse(y, predictions))
print("R²:", model.r2_score(y, predictions))
```

```
# New prediction

new_experience = np.array([6])

new_salary = model.predict(new_experience)

print("Predicted salary:", new_salary)
```

---

13. What is happening internally?

The most important part is this:

```
y_pred = self.w * X + self.b
```

The model makes predictions.

Then:

```
error = y_pred - y
```

It checks how far the prediction is from the actual value.

Then:

```
dw = (2 / n) * np.sum(X * error)
db = (2 / n) * np.sum(error)
```

It calculates how the parameters need to change.

Finally:

```
self.w = self.w - self.learning_rate * dw
self.b = self.b - self.learning_rate * db
```

The model learns.

This happens thousands of times:

```
Initial
w = 0
b = 0

       ↓

Prediction
```

```
        ↓
Calculate error
        ↓
Calculate gradients
        ↓
Update w and b
        ↓
Prediction
        ↓
Calculate error
        ↓
Update w and b
        ↓
         ...
        ↓
Final w and b
```

---

14. Why learning rate matters

Suppose:

```
learning_rate = 0.0001
```

Learning will be very slow.

If:

```
learning_rate = 0.01
```

Learning can be reasonable.

If:

```
learning_rate = 1
```

the model may overshoot the minimum and fail to converge.

So:

```
Too small → slow learning

Good value → convergence

Too large → unstable/diverges
```

---

15. Linear Regression with multiple features

Real ML projects usually have multiple features.

For example, predicting salary from:

```
experience
education
projects
skills
```

The equation becomes:

$$\hat{y} = w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b$$

In matrix notation:

$$\hat{y} = XW + b$$

Python:

```
y_pred = np.dot(X, W) + b
```

The gradient descent concept remains the same.

---

16. Important concepts to learn for ML interviews

For a strong understanding of Linear Regression, learn these in this order:

1. **Dependent and independent variables**

2. Simple Linear Regression
3. Multiple Linear Regression
4. Cost Function
5. MSE
6. Gradient Descent
7. Learning Rate
8. Weights and Bias
9. Normal Equation
10. R^2 Score
11. Adjusted R^2
12. MAE
13. RMSE
14. Residuals
15. Underfitting
16. Overfitting
17. Feature Scaling
18. Train/Test Split
19. Regularization
20. Ridge Regression
21. Lasso Regression
22. Multicollinearity
23. Assumptions of Linear Regression

The key idea

If you understand this loop:

```
X
↓
Prediction
↓
Error
↓
Cost
↓
Gradient
↓
Update W/B
↓
Repeat
```

you understand the **core mechanism behind training Linear Regression with Gradient Descent**.

You said:

I need to build above linear regression model using pyspark

ChatGPT said:

Yes. If you want to build the **same Linear Regression model using PySpark**, there are two useful approaches:

1. **PySpark MLlib LinearRegression** — production/standard approach.
2. **Linear Regression from scratch using PySpark DataFrame operations** — best for understanding how the algorithm works.

Since you asked for the model development from scratch, let's build **the second approach first**, then compare it with MLlib.

---

1. Install / start PySpark

```
pip install pyspark
```

Check:

```
pyspark --version
```

---

2. Create a PySpark DataFrame

We'll use the same example:

> Predict salary based on years of experience.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("LinearRegressionFromScratch") \
    .getOrCreate()


data = [
    (1.0, 35.0),
    (2.0, 40.0),
    (3.0, 50.0),
    (4.0, 60.0),
    (5.0, 65.0)
]
```

```
df = spark.createDataFrame(
    data,
    ["experience", "salary"]
)

df.show()
```

Output:

```
+----------+------+
|experience|salary|
+----------+------+
|       1.0|  35.0|
|       2.0|  40.0|
|       3.0|  50.0|
|       4.0|  60.0|
|       5.0|  65.0|
+----------+------+
```

---

3. Understand the model

Our model is:

$$\hat{y} = wx + b$$

where:

```
x = experience
y = salary
w = weight
b = bias
```

Initially:

```
w = 0.0
b = 0.0
```

---

4. Prediction using PySpark

We can calculate predictions using Spark DataFrame expressions.

```
from pyspark.sql.functions import col, lit

w = 0.0
b = 0.0

prediction_df = df.withColumn(
    "prediction",
    w * col("experience") + b
)

prediction_df.show()
```

Initially:

```
+----------+------+----------+
|experience|salary|prediction|
+----------+------+----------+
|       1.0|  35.0|       0.0|
|       2.0|  40.0|       0.0|
|       3.0|  50.0|       0.0|
|       4.0|  60.0|       0.0|
|       5.0|  65.0|       0.0|
+----------+------+----------+
```

---

5. Calculate the error

The error is:

$$error = \hat{y} - y$$

In PySpark:

```
prediction_df = prediction_df.withColumn(
    "error",
    col("prediction") - col("salary")
)

prediction_df.show()
```

---

6. Calculate MSE

Our cost function is:

$$MSE = \frac{1}{n}\sum(\hat{y} - y)^2$$

PySpark:

```
from pyspark.sql.functions import avg

mse = prediction_df \
    .select(
        avg(
            (col("prediction") - col("salary")) ** 2
        ).alias("mse")
    ) \
    .collect()[0]["mse"]

print("MSE:", mse)
```

---

7. Calculate gradients

For one feature:

$$\frac{\partial MSE}{\partial w} = \frac{2}{n}\sum x(\hat{y} - y)$$

and:

$$\frac{\partial MSE}{\partial b} = \frac{2}{n}\sum(\hat{y} - y)$$

In PySpark:

```
from pyspark.sql.functions import sum as spark_sum

n = df.count()

gradient_df = prediction_df.select(
    spark_sum(
        col("experience") * col("error")
```

```
    ).alias("dw_sum"),

    spark_sum(
        col("error")
    ).alias("db_sum")
)

gradients = gradient_df.collect()[0]

dw = (2 / n) * gradients["dw_sum"]

db = (2 / n) * gradients["db_sum"]

print("dw:", dw)
print("db:", db)
```

---

8. Update the model

Gradient descent:

$$w = w - \alpha dw$$

$$b = b - \alpha db$$

Python:

```
learning_rate = 0.01

w = w - learning_rate * dw

b = b - learning_rate * db

print("w:", w)
print("b:", b)
```

This is **one iteration**.

We need to repeat this process.

---

9. Build the training loop

Now combine everything.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg, sum as spark_sum


spark = SparkSession.builder \
    .appName("LinearRegressionFromScratch") \
    .getOrCreate()


data = [
    (1.0, 35.0),
    (2.0, 40.0),
    (3.0, 50.0),
    (4.0, 60.0),
    (5.0, 65.0)
]


df = spark.createDataFrame(
    data,
    ["experience", "salary"]
)


# Model parameters
w = 0.0
b = 0.0

learning_rate = 0.01
epochs = 1000

n = df.count()


for epoch in range(epochs):

    # Prediction
    prediction_df = df.withColumn(
        "prediction",
        w * col("experience") + b
    )
```

```
    # Error
    prediction_df = prediction_df.withColumn(
        "error",
        col("prediction") - col("salary")
    )

    # Gradients
    gradient = prediction_df.select(
        spark_sum(
            col("experience") * col("error")
        ).alias("dw"),

        spark_sum(
            col("error")
        ).alias("db")
    ).collect()[0]

    dw = (2 / n) * gradient["dw"]

    db = (2 / n) * gradient["db"]

    # Update parameters
    w = w - learning_rate * dw

    b = b - learning_rate * db


print("Final weight:", w)
print("Final bias:", b)
```

This is a **Linear Regression implementation using PySpark DataFrame operations without using MLlib's LinearRegression algorithm**.

---

10. Make predictions

After training:

```
result = df.withColumn(
    "prediction",
    w * col("experience") + b
)

result.show()
```

You should get predictions approximately like:

```
+----------+------+------------------+
|experience|salary|prediction        |
+----------+------+------------------+
|       1.0|  35.0|~34               |
|       2.0|  40.0|~42               |
|       3.0|  50.0|~50               |
|       4.0|  60.0|~58               |
|       5.0|  65.0|~66               |
+----------+------+------------------+
```

The exact values depend on the number of iterations and learning rate.

---

11. Calculate R²

Now add the R² calculation.

$$R^2 = 1 - \frac{SS_{res}}{SS_{total}}$$

```
from pyspark.sql.functions import avg

mean_y = df.select(
    avg("salary").alias("mean")
).collect()[0]["mean"]

result = result.withColumn(
    "residual_squared",
    (col("salary") - col("prediction")) ** 2
)

result = result.withColumn(
    "total_squared",
    (col("salary") - mean_y) ** 2
)

values = result.select(
    spark_sum("residual_squared").alias("ss_res"),
    spark_sum("total_squared").alias("ss_total")
).collect()[0]

r2 = 1 - (
```

```
    values["ss_res"] / values["ss_total"]
)

print("R²:", r2)
```

---

12. Complete PySpark implementation

You can put everything into a reusable class.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col,
    avg,
    sum as spark_sum
)


class LinearRegressionFromScratch:

    def __init__(
        self,
        learning_rate=0.01,
        epochs=1000
    ):

        self.learning_rate = learning_rate
        self.epochs = epochs

        self.w = 0.0
        self.b = 0.0


    def fit(self, df, feature, target):

        n = df.count()

        for epoch in range(self.epochs):

            # Prediction
            temp = df.withColumn(
                "prediction",
                self.w * col(feature) + self.b
            )
```

```
        # Error
        temp = temp.withColumn(
            "error",
            col("prediction") - col(target)
        )

        # Gradient
        gradient = temp.select(

            spark_sum(
                col(feature) * col("error")
            ).alias("dw"),

            spark_sum(
                col("error")
            ).alias("db")

        ).collect()[0]


        dw = (2 / n) * gradient["dw"]

        db = (2 / n) * gradient["db"]


        # Update
        self.w -= self.learning_rate * dw

        self.b -= self.learning_rate * db


    def predict(self, df, feature):

        return df.withColumn(
            "prediction",
            self.w * col(feature) + self.b
        )


    def evaluate(self, df, feature, target):

        result = self.predict(df, feature)

        # MSE
```

```
        mse = result.select(
            avg(
                (col("prediction") - col(target)) ** 2
            ).alias("mse")
        ).collect()[0]["mse"]


        # Mean target
        mean_y = df.select(
            avg(target).alias("mean")
        ).collect()[0]["mean"]


        # SS Residual
        ss_res = result.select(
            spark_sum(
                (col(target) - col("prediction")) ** 2
            ).alias("value")
        ).collect()[0]["value"]


        # SS Total
        ss_total = df.select(
            spark_sum(
                (col(target) - mean_y) ** 2
            ).alias("value")
        ).collect()[0]["value"]


        r2 = 1 - (ss_res / ss_total)


        return mse, r2


spark = SparkSession.builder \
    .appName("LinearRegression") \
    .getOrCreate()


data = [
    (1.0, 35.0),
    (2.0, 40.0),
    (3.0, 50.0),
    (4.0, 60.0),
```

```
    (5.0, 65.0)
]


df = spark.createDataFrame(
    data,
    ["experience", "salary"]
)


model = LinearRegressionFromScratch(
    learning_rate=0.01,
    epochs=1000
)


model.fit(
    df,
    "experience",
    "salary"
)


print("Weight:", model.w)

print("Bias:", model.b)


predictions = model.predict(
    df,
    "experience"
)

predictions.show()


mse, r2 = model.evaluate(
    df,
    "experience",
    "salary"
)


print("MSE:", mse)
```

```
print("R²:", r2)
```

---

13. But there is an important PySpark point

For **learning**, the above implementation is excellent.

For a real Spark project, however, you normally shouldn't repeatedly call:

```
.collect()
```

inside thousands of iterations.

Why?

Because:

```
Driver
   ↓
collect()
   ↓
Workers → Driver
   ↓
Update parameters
   ↓
Workers
```

This can become expensive with a large dataset.

For production ML, PySpark provides **MLlib**.

---

14. Production approach — PySpark MLlib

The standard PySpark implementation is much shorter.

```
from pyspark.sql import SparkSession

from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression


spark = SparkSession.builder \
    .appName("LinearRegressionMLlib") \
    .getOrCreate()
```

```
data = [
    (1.0, 35.0),
    (2.0, 40.0),
    (3.0, 50.0),
    (4.0, 60.0),
    (5.0, 65.0)
]


df = spark.createDataFrame(
    data,
    ["experience", "salary"]
)
```

Convert the feature into Spark's vector format:

```
assembler = VectorAssembler(
    inputCols=["experience"],
    outputCol="features"
)

data = assembler.transform(df)

data.show()
```

You get:

```
+----------+------+--------+
|experience|salary|features|
+----------+------+--------+
|       1.0|  35.0|   [1.0]|
|       2.0|  40.0|   [2.0]|
|       3.0|  50.0|   [3.0]|
|       4.0|  60.0|   [4.0]|
|       5.0|  65.0|   [5.0]|
+----------+------+--------+
```

Train:

```
lr = LinearRegression(
    featuresCol="features",
    labelCol="salary"
)

model = lr.fit(data)
```

Get coefficients:

```
print("Coefficient:", model.coefficients)

print("Intercept:", model.intercept)
```

Predict:

```
predictions = model.transform(data)

predictions.select(
    "experience",
    "salary",
    "prediction"
).show()
```

Evaluate:

```
print("RMSE:", model.summary.rootMeanSquaredError)

print("R²:", model.summary.r2)
```

---

15. From-scratch vs MLlib

Feature	From Scratch	MLlib
Understand algorithm	⭐⭐⭐⭐⭐	⭐⭐⭐
Gradient Descent understanding	Yes	Hidden
Production use	Limited	⭐⭐⭐⭐⭐
Distributed Spark	Partially	Yes
Easy to implement	Medium	Very easy
Interview explanation	Excellent	Excellent
Large dataset	Not ideal	Excellent
Hyperparameters	Manual	Built-in
Pipeline support	Manual	Yes

Recommended learning path

For your **PySpark + Data Engineering/ML learning**, I would learn it in this order:

```
Python Linear Regression
        ↓
NumPy Linear Regression
        ↓
PySpark DataFrame Linear Regression
        ↓
PySpark MLlib LinearRegression
        ↓
Train/Test Split
        ↓
Feature Engineering
        ↓
Model Evaluation
        ↓
ML Pipeline
        ↓
Cross Validation
        ↓
Hyperparameter Tuning
        ↓
Save/Load Model
        ↓
Deploy Model
```

The most important next step is **multiple linear regression in PySpark**, where we use several features such as `experience`, `skills`, `projects`, and `education`, implement the vector/matrix calculations, and then build the same model using `pyspark.ml`.

You said:

I have css with below columns now I need to build linear regression model using pyspark
['pool_row', 'inchikey14', 'pubchem_cid', 'smiles', 'formula', 'monoisotopic_mass', 'log1p_substances', 'log1p_pubmed', 'log1p_patents', 'pop', 'substances_approx', 'pubmed_approx', 'patents_approx', 'np_coconut', 'np_lotus', 'np_npatlas']

ChatGPT said:

Yes. With these columns, the first thing we need to decide is **what you want to predict**. Since you have several numerical columns representing chemical popularity/database evidence, a reasonable first Linear Regression target is `np_coconut`, using the other numeric columns as predictors.

However, `pool_row`, `inchikey14`, `smiles`, and `formula` are identifiers/text and should **not** be directly fed into Linear Regression.

A good starting design is:

```
CSS data
   ↓
Load CSV with PySpark
   ↓
Select numerical features
   ↓
Handle nulls
   ↓
Assemble features
   ↓
Train/Test split
   ↓
Linear Regression
   ↓
Prediction
   ↓
RMSE / MAE / R²
   ↓
Feature importance
```

1. Your columns

You have:

```
columns = [
    'pool_row',
    'inchikey14',
    'pubchem_cid',
    'smiles',
    'formula',
    'monoisotopic_mass',
    'log1p_substances',
    'log1p_pubmed',
    'log1p_patents',
    'pop',
    'substances_approx',
    'pubmed_approx',
    'patents_approx',
    'np_coconut',
    'np_lotus',
```

```
    'np_npatlas'
]
```

I would initially use:

Target

```
np_coconut
```

Features

```
monoisotopic_mass
log1p_substances
log1p_pubmed
log1p_patents
pop
substances_approx
pubmed_approx
patents_approx
np_lotus
np_npatlas
```

And exclude:

```
pool_row
inchikey14
pubchem_cid
smiles
formula
```

because they are identifiers/string representations rather than suitable continuous predictors.

---

2. Complete PySpark Linear Regression

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression
from pyspark.ml.evaluation import RegressionEvaluator


# ------------------------------------------------
# 1. Create Spark session
# ------------------------------------------------
```

```
spark = SparkSession.builder \
    .appName("ChemicalLinearRegression") \
    .getOrCreate()


# ------------------------------------------------
# 2. Load CSS dataset
# ------------------------------------------------

df = spark.read.csv(
    "css.csv",
    header=True,
    inferSchema=True
)


# ------------------------------------------------
# 3. Display schema
# ------------------------------------------------

df.printSchema()

df.show(5)
```

---

3. Select features

```
feature_columns = [
    "monoisotopic_mass",
    "log1p_substances",
    "log1p_pubmed",
    "log1p_patents",
    "pop",
    "substances_approx",
    "pubmed_approx",
    "patents_approx",
    "np_lotus",
    "np_npatlas"
]

target_column = "np_coconut"
```

So our mathematical model becomes approximately:

$$np_coconut = \beta_0 + \beta_1(monoi...) + \beta_2(log1p_substances) + ... + \beta_{10}(np_npatlas)$$

---

4. Check data types

This is important because your CSV may contain numeric values stored as strings.

```
df.select(
    feature_columns + [target_column]
).printSchema()
```

You want something similar to:

```
monoisotopic_mass     double
log1p_substances      double
log1p_pubmed          double
log1p_patents         double
pop                   double
substances_approx     double
pubmed_approx         double
patents_approx        double
np_lotus              double
np_npatlas            double
np_coconut            double
```

---

5. Handle NULL values

Check:

```
from pyspark.sql.functions import count, when, isnan

for column in feature_columns + [target_column]:

    df.select(
        count(
            when(
                col(column).isNull() |
                isnan(col(column)),
                column
            )
        ).alias(column)
    ).show()
```

For a simple model, we can remove rows with missing values:

```
model_df = df.select(
    feature_columns + [target_column]
).dropna()
```

Check:

```
print("Original rows:", df.count())

print("Rows after cleaning:", model_df.count())
```

---

6. Assemble features

Spark's ML algorithms expect all features in a single vector.

```
assembler = VectorAssembler(
    inputCols=feature_columns,
    outputCol="features"
)

assembled_df = assembler.transform(model_df)
```

Now:

```
assembled_df.select(
    "features",
    target_column
).show(5, truncate=False)
```

You'll see something like:

```
+-------------------------------------------------+----------+
|features                                         |np_coconut|
+-------------------------------------------------+----------+
|[...,...,...,...,...,...,...,...,...,...]        |...       |
|[...,...,...,...,...,...,...,...,...,...]        |...       |
+-------------------------------------------------+----------+
```

---

7. Create train/test data

Use an 80/20 split:

```
train_df, test_df = assembled_df.randomSplit(
    [0.8, 0.2],
    seed=42
)

print("Training:", train_df.count())
print("Testing :", test_df.count())
```

The `seed=42` makes the split reproducible.

---

8. Build Linear Regression model

```
lr = LinearRegression(
    featuresCol="features",
    labelCol=target_column,
    predictionCol="prediction"
)
```

Train:

```
model = lr.fit(train_df)
```

Now your model has learned:

```
weights/coefficient
+
intercept
```

---

9. Get coefficients

```
print("Intercept:", model.intercept)

print("Coefficients:", model.coefficients)
```

But coefficients by themselves are difficult to interpret.

Let's associate each coefficient with its feature.

```
for feature, coefficient in zip(
    feature_columns,
    model.coefficients
):
```

```
    print(
        f"{feature:25} {coefficient}"
    )
```

Example output:

```
monoisotopic_mass         0.00123
log1p_substances          0.423
log1p_pubmed              0.182
log1p_patents             0.091
pop                       0.552
substances_approx         0.003
pubmed_approx             0.002
patents_approx            0.001
np_lotus                  0.723
np_npatlas                0.342
```

These are illustrative values; your actual coefficients will depend on your CSS dataset.

---

10. Make predictions

```
predictions = model.transform(test_df)
```

View:

```
predictions.select(
    target_column,
    "prediction"
).show(20)
```

For example:

```
+----------+------------------+
|np_coconut|prediction        |
+----------+------------------+
|10.5      |10.82             |
|15.2      |14.91             |
|20.1      |20.43             |
+----------+------------------+
```

---

11. Evaluate the model

RMSE

```
rmse_evaluator = RegressionEvaluator(
    labelCol=target_column,
    predictionCol="prediction",
    metricName="rmse"
)

rmse = rmse_evaluator.evaluate(predictions)

print("RMSE:", rmse)
```

MAE

```
mae_evaluator = RegressionEvaluator(
    labelCol=target_column,
    predictionCol="prediction",
    metricName="mae"
)

mae = mae_evaluator.evaluate(predictions)

print("MAE:", mae)
```

R^2

```
r2_evaluator = RegressionEvaluator(
    labelCol=target_column,
    predictionCol="prediction",
    metricName="r2"
)

r2 = r2_evaluator.evaluate(predictions)

print("R²:", r2)
```

---

12. Complete program

You can start with this single script:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression
from pyspark.ml.evaluation import RegressionEvaluator


# ================================================
# 1. Spark Session
# ================================================

spark = SparkSession.builder \
    .appName("CSSLinearRegression") \
    .getOrCreate()


# ================================================
# 2. Load CSS data
# ================================================

df = spark.read.csv(
    "css.csv",
    header=True,
    inferSchema=True
)


# ================================================
# 3. Features and target
# ================================================

feature_columns = [
    "monoisotopic_mass",
    "log1p_substances",
    "log1p_pubmed",
    "log1p_patents",
    "pop",
    "substances_approx",
    "pubmed_approx",
    "patents_approx",
    "np_lotus",
    "np_npatlas"
]
```

```
target_column = "np_coconut"


# ================================================
# 4. Select and clean data
# ================================================

model_df = df.select(
    feature_columns + [target_column]
).dropna()


print("Total records:", model_df.count())


# ================================================
# 5. Assemble features
# ================================================

assembler = VectorAssembler(
    inputCols=feature_columns,
    outputCol="features"
)

assembled_df = assembler.transform(model_df)


# ================================================
# 6. Train/Test split
# ================================================

train_df, test_df = assembled_df.randomSplit(
    [0.8, 0.2],
    seed=42
)


print("Training records:", train_df.count())
print("Testing records :", test_df.count())


# ================================================
# 7. Create Linear Regression
# ================================================
```

```
lr = LinearRegression(
    featuresCol="features",
    labelCol=target_column,
    predictionCol="prediction"
)



# ================================================
# 8. Train
# ================================================

model = lr.fit(train_df)



# ================================================
# 9. Model parameters
# ================================================

print("\n========== MODEL ==========")

print("Intercept:", model.intercept)

print("\nCoefficients:")

for feature, coefficient in zip(
    feature_columns,
    model.coefficients
):

    print(
        f"{feature:25} {coefficient}"
    )



# ================================================
# 10. Prediction
# ================================================

predictions = model.transform(test_df)



print("\n========== PREDICTIONS ==========")

predictions.select(
    target_column,
```

```
    "prediction"
).show(20)


# ================================================
# 11. RMSE
# ================================================

rmse = RegressionEvaluator(
    labelCol=target_column,
    predictionCol="prediction",
    metricName="rmse"
).evaluate(predictions)


# ================================================
# 12. MAE
# ================================================

mae = RegressionEvaluator(
    labelCol=target_column,
    predictionCol="prediction",
    metricName="mae"
).evaluate(predictions)


# ================================================
# 13. R2
# ================================================

r2 = RegressionEvaluator(
    labelCol=target_column,
    predictionCol="prediction",
    metricName="r2"
).evaluate(predictions)


print("\n========== MODEL EVALUATION ==========")

print("RMSE :", rmse)

print("MAE  :", mae)

print("R²   :", r2)
```

```
spark.stop()
```

---

13. One important issue with your CSS data

Before treating this as the final model, I would **not blindly use all 10 numerical columns**.

For example:

```
np_coconut
np_lotus
np_npatlas
```

appear to be related natural-product databases/sources. Likewise:

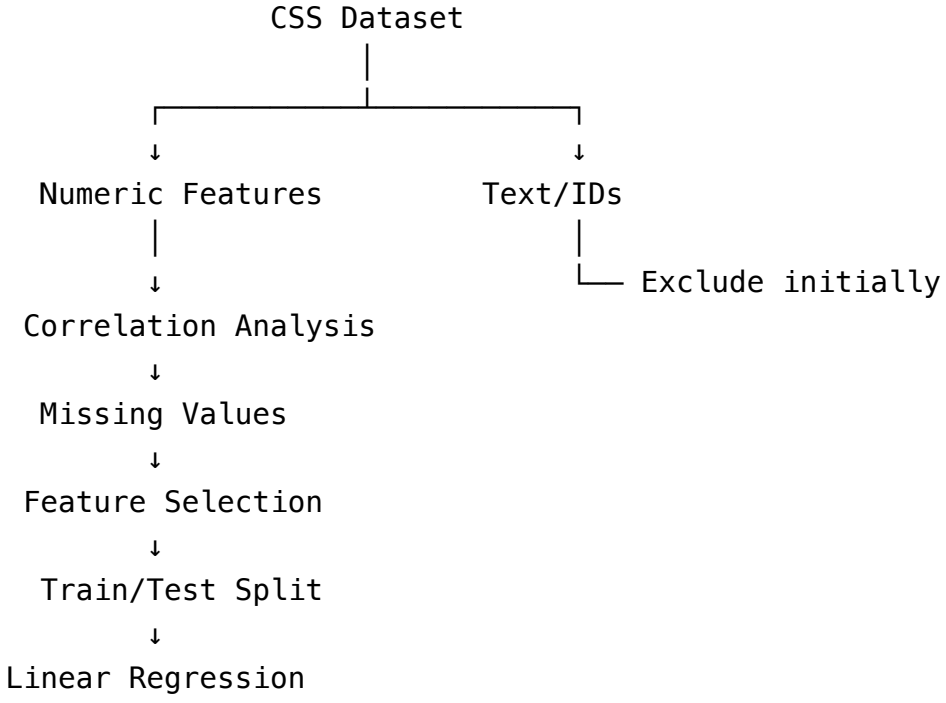
```
substances_approx
pubmed_approx
patents_approx
```

are likely derived/approximate versions of:

```
log1p_substances
log1p_pubmed
log1p_patents
```

That can create **feature leakage or strong multicollinearity** depending on how these columns were generated.

So your next analysis should be:



```
          ↓
   RMSE / MAE / R²
          ↓
   Residual Analysis
          ↓
  Multicollinearity Check
          ↓
      Final Model
```

Most importantly, we need to establish what `np_coconut` represents and whether it is actually the target you want to predict. If your actual objective is to predict `pop`, `np_lotus`, `np_npatlas`, or another column, simply changing the target can produce a completely different and more meaningful model.

If you give me **5–10 sample rows of your CSS data (`df.show(10, truncate=False)`) and tell me which column you want to predict**, I can build the complete PySpark pipeline specifically for your dataset, including **EDA → correlation → feature selection → train/test → Linear Regression → RMSE/MAE/R² → coefficient analysis → prediction**.

You said:

In above dataset all columns come as string while vetrization following below error is coming
Data type string of column log1p_substances is not supported.
Data type string of column log1p_pubmed is not supported.
Data type string of column log1p_patents is not supported.
Data type string of column pop is not supported.
Data type string of column substances_approx is not supported.
Data type string of column pubmed_approx is not supported.
Data type string of column patents_approx is not supported.
Data type string of column np_lotus is not supported.
Data type string of column np_npatlas is not supported.

ChatGPT said:

Yes — the problem is that Spark has read your CSV columns as **`string`**, while `VectorAssembler` requires numeric types such as `double` or `float`.

The clean solution is to **explicitly cast the ML columns to `double` before `VectorAssembler`**.

1. Check your schema

```
df.printSchema()
```

You are probably seeing:

```
root
 |-- pool_row: string
```

```
 |-- inchikey14: string
 |-- pubchem_cid: string
 |-- smiles: string
 |-- formula: string
 |-- monoisotopic_mass: string
 |-- log1p_substances: string
 |-- log1p_pubmed: string
 |-- log1p_patents: string
 |-- pop: string
 |-- substances_approx: string
 |-- pubmed_approx: string
 |-- patents_approx: string
 |-- np_coconut: string
 |-- np_lotus: string
 |-- np_npatlas: string
```

---

2. Cast all ML columns to `double`

I recommend doing this explicitly rather than relying on `inferSchema`.

```
from pyspark.sql.functions import col

numeric_columns = [
    "monoisotopic_mass",
    "log1p_substances",
    "log1p_pubmed",
    "log1p_patents",
    "pop",
    "substances_approx",
    "pubmed_approx",
    "patents_approx",
    "np_coconut",
    "np_lotus",
    "np_npatlas"
]

for column in numeric_columns:
    df = df.withColumn(
        column,
        col(column).cast("double")
    )
```

Now check:

```
df.printSchema()
```

You should get:

```
 |-- monoisotopic_mass: double
 |-- log1p_substances: double
 |-- log1p_pubmed: double
 |-- log1p_patents: double
 |-- pop: double
 |-- substances_approx: double
 |-- pubmed_approx: double
 |-- patents_approx: double
 |-- np_coconut: double
 |-- np_lotus: double
 |-- np_npatlas: double
```

---

3. Important: handle invalid strings

There may be values such as:

```
"NA"
"null"
""
"N/A"
```

When you cast these:

```
col(column).cast("double")
```

Spark will convert invalid numeric strings to `NULL`.

Therefore, after casting:

```
df = df.dropna(
    subset=numeric_columns
)
```

Check:

```
print("Rows:", df.count())
```

---

4. Now use VectorAssembler

For example, if your target is `np_coconut`:

```
from pyspark.ml.feature import VectorAssembler

feature_columns = [
    "monoisotopic_mass",
    "log1p_substances",
    "log1p_pubmed",
    "log1p_patents",
    "pop",
    "substances_approx",
    "pubmed_approx",
    "patents_approx",
    "np_lotus",
    "np_npatlas"
]

assembler = VectorAssembler(
    inputCols=feature_columns,
    outputCol="features"
)

assembled_df = assembler.transform(df)
```

Then:

```
assembled_df.select(
    "features",
    "np_coconut"
).show(5, truncate=False)
```

You should now see:

```
+-----------------------------------------------------+----------+
|features                                             |np_coconut|
+-----------------------------------------------------+----------+
|[12.3,4.2,2.1,1.3,0.8,100.0,20.0,10.0,5.0,3.0] |5.0       |
|[...]                                                |...       |
+-----------------------------------------------------+----------+
```

---

5. Complete corrected code

I would structure your code like this:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col
from pyspark.ml.feature import VectorAssembler
```

```
from pyspark.ml.regression import LinearRegression
from pyspark.ml.evaluation import RegressionEvaluator


# ====================================================
# 1. Spark Session
# ====================================================

spark = SparkSession.builder \
    .appName("CSS_Linear_Regression") \
    .getOrCreate()


# ====================================================
# 2. Read CSV
# ====================================================

df = spark.read.csv(
    "css.csv",
    header=True,
    inferSchema=False
)


# ====================================================
# 3. Numeric columns
# ====================================================

numeric_columns = [
    "monoisotopic_mass",
    "log1p_substances",
    "log1p_pubmed",
    "log1p_patents",
    "pop",
    "substances_approx",
    "pubmed_approx",
    "patents_approx",
    "np_coconut",
    "np_lotus",
    "np_npatlas"
]


# ====================================================
# 4. Convert String → Double
```

```
# ====================================================

for column in numeric_columns:

    df = df.withColumn(
        column,
        col(column).cast("double")
    )


# ====================================================
# 5. Check schema
# ====================================================

df.printSchema()


# ====================================================
# 6. Remove invalid/null values
# ====================================================

df = df.dropna(
    subset=numeric_columns
)


# ====================================================
# 7. Define features
# ====================================================

feature_columns = [
    "monoisotopic_mass",
    "log1p_substances",
    "log1p_pubmed",
    "log1p_patents",
    "pop",
    "substances_approx",
    "pubmed_approx",
    "patents_approx",
    "np_lotus",
    "np_npatlas"
]


target_column = "np_coconut"
```

```
# ======================================================
# 8. Vectorization
# ======================================================

assembler = VectorAssembler(
    inputCols=feature_columns,
    outputCol="features"
)

assembled_df = assembler.transform(df)


# ======================================================
# 9. Select ML columns
# ======================================================

final_df = assembled_df.select(
    "features",
    target_column
)


final_df.show(
    5,
    truncate=False
)


# ======================================================
# 10. Train/Test Split
# ======================================================

train_df, test_df = final_df.randomSplit(
    [0.8, 0.2],
    seed=42
)


# ======================================================
# 11. Linear Regression
# ======================================================

lr = LinearRegression(
```

```
    featuresCol="features",
    labelCol=target_column,
    predictionCol="prediction"
)


# ======================================================
# 12. Train
# ======================================================

model = lr.fit(train_df)


# ======================================================
# 13. Prediction
# ======================================================

predictions = model.transform(test_df)


predictions.select(
    target_column,
    "prediction"
).show(20)


# ======================================================
# 14. Evaluation
# ======================================================

rmse = RegressionEvaluator(
    labelCol=target_column,
    predictionCol="prediction",
    metricName="rmse"
).evaluate(predictions)


mae = RegressionEvaluator(
    labelCol=target_column,
    predictionCol="prediction",
    metricName="mae"
).evaluate(predictions)


r2 = RegressionEvaluator(
```

```
    labelCol=target_column,
    predictionCol="prediction",
    metricName="r2"
).evaluate(predictions)


print("================================")
print("Linear Regression Results")
print("================================")

print("RMSE :", rmse)
print("MAE  :", mae)
print("R²   :", r2)

print("================================")
print("Intercept")
print("================================")

print(model.intercept)


print("================================")
print("Coefficients")
print("================================")

for feature, coefficient in zip(
    feature_columns,
    model.coefficients
):

    print(
        feature,
        "=>",
        coefficient
    )


spark.stop()
```

Even better: cast during CSV loading

You can also define a schema while reading the CSV, which is cleaner for a production pipeline:

```
from pyspark.sql.types import (
    StructType,
```

```
    StructField,
    StringType,
    DoubleType
)

schema = StructType([
    StructField("pool_row", StringType(), True),
    StructField("inchikey14", StringType(), True),
    StructField("pubchem_cid", StringType(), True),
    StructField("smiles", StringType(), True),
    StructField("formula", StringType(), True),

    StructField("monoisotopic_mass", DoubleType(), True),
    StructField("log1p_substances", DoubleType(), True),
    StructField("log1p_pubmed", DoubleType(), True),
    StructField("log1p_patents", DoubleType(), True),
    StructField("pop", DoubleType(), True),

    StructField("substances_approx", DoubleType(), True),
    StructField("pubmed_approx", DoubleType(), True),
    StructField("patents_approx", DoubleType(), True),

    StructField("np_coconut", DoubleType(), True),
    StructField("np_lotus", DoubleType(), True),
    StructField("np_npatlas", DoubleType(), True)
])

df = spark.read.csv(
    "css.csv",
    header=True,
    schema=schema
)
```

This is preferable because Spark knows the correct types **from the beginning**, rather than loading everything as strings and converting later.

One more thing to watch

Because your dataset contains both:

```
log1p_substances
substances_approx
```

and similarly:

```
log1p_pubmed       ↔ pubmed_approx
log1p_patents      ↔ patents_approx
```

I would **not immediately assume all of them should be features together**. They may be highly correlated or derived from one another. After fixing the datatype issue, the next useful step is to run **correlation analysis and feature selection before training the final Linear Regression model**.

You said:

explain the below model output
RMSE : 0.4003069211203466
MAE : 0.3518789086750922
R² : 0.29469294131883683

ChatGPT said:

These results tell you that your Linear Regression model is making **reasonably small prediction errors, but it explains only a limited portion of the variation in `np_coconut`**.

Your output is:

```
RMSE : 0.4003
MAE  : 0.3519
R²   : 0.2947
```

Assuming your target is `np_coconut`, let's understand each metric.

1. MAE = 0.3519

Mean Absolute Error tells you the average absolute difference between the actual and predicted values.

$$MAE = \frac{1}{n}\sum |y - \hat{y}|$$

Your:

```
MAE = 0.3519
```

means:

> On average, the model's prediction is approximately **0.352 units away from the actual `np_coconut` value**.

For example, if the actual values were:

```
Actual      Predicted
5.0         5.2
4.0         3.7
6.0         5.8
```

the prediction errors are approximately:

```
0.2
0.3
0.2
```

So MAE summarizes those errors.

Lower MAE is better.

---

2. RMSE = 0.4003

RMSE is **Root Mean Squared Error**:

$$RMSE = \sqrt{\frac{1}{n}\sum(y - \hat{y})^2}$$

Your:

```
RMSE = 0.4003
```

means the typical prediction error, with larger errors penalized more strongly, is around **0.40 units**.

Notice:

```
MAE  = 0.3519
RMSE = 0.4003
```

RMSE is larger than MAE, which is normal.

The difference suggests that there are some prediction errors larger than the average error, because RMSE gives more weight to larger mistakes.

Lower RMSE is better.

---

3. R² = 0.2947

This is probably the **most important result to investigate**.

Your:

```
R² = 0.2947
```

means approximately:

$$29.47\%$$

of the variation in `np_coconut` is explained by the features you provided to the model.

Approximately:

```
29.47% → explained by the model
70.53% → not explained by the model
```

So your model currently has **limited explanatory/predictive power**.

A simplified interpretation:

> If `np_coconut` varies substantially across your chemical records, your current features explain only about 29.5% of that variation.

---

4. Is $R^2 = 0.29$ good or bad?

There isn't a universal rule such as:

```
R² > 0.8 = good
R² < 0.5 = bad
```

It depends heavily on the problem.

For your chemical/natural-product dataset, an R^2 of `0.2947` could be useful as a baseline, but I would **not call this a strong Linear Regression model yet**.

I'd classify it as:

```
R² = 0.29

        ↓

Weak-to-moderate explanatory power

        ↓

Good baseline

        ↓

Needs feature investigation/improvement
```

---

5. Your model in one sentence

Your results can be explained in an ML report as:

> **The Linear Regression model achieved an RMSE of 0.4003 and MAE of 0.3519, indicating relatively small average prediction errors, while an R^2 of 0.2947 indicates that the model explains approximately 29.47% of the variance in the `np_coconut` target variable. Therefore, the model provides a baseline level of predictive performance but has substantial unexplained variation that may require additional features, feature engineering, or a different regression algorithm.**

---

6. One thing I would check before improving the model

Your dataset has these related columns:

```
log1p_substances
substances_approx

log1p_pubmed
pubmed_approx

log1p_patents
patents_approx
```

These may contain overlapping information.

You also have:

```
np_lotus
np_npatlas
```

which may have relationships with:

```
np_coconut
```

So before simply adding more algorithms, I would investigate:

```
            Current Model
                  ↓
          Correlation Matrix
                  ↓
        Feature Relationships
```

```
                ↓
     Multicollinearity (VIF)
                ↓
     Outlier Analysis
                ↓
    Feature Engineering
                ↓
    Linear Regression v2
                ↓
     Compare R² / RMSE / MAE
```

Especially important

Don't judge the model from **R² alone**. You should compare it against a baseline and ideally inspect the target's scale/distribution.

For example, if `np_coconut` typically ranges from `0` to `2`, then an MAE of `0.35` could be significant. If it ranges from `0` to `100`, the same MAE would be extremely small.

If you show me the output of:

```
df.select("np_coconut").describe().show()
```

I can tell you **whether RMSE = 0.40 and MAE = 0.35 are actually good for your specific `np_coconut` data**, rather than judging them in isolation.

You said:

for the same dataset I want to build decision tree as separate cell

ChatGPT said:

Yes. You can build the **Decision Tree Regression model as a separate PySpark cell** using the same CSS dataset and the same target (`np_coconut`).

Cell 1 — Prepare numeric columns

If your previous cell already converted the columns to `double` and removed nulls, you can skip this cell.

```
from pyspark.sql.functions import col

numeric_columns = [
    "monoisotopic_mass",
    "log1p_substances",
    "log1p_pubmed",
    "log1p_patents",
    "pop",
```

```
    "substances_approx",
    "pubmed_approx",
    "patents_approx",
    "np_lotus",
    "np_npatlas",
    "np_coconut"
]

for c in numeric_columns:
    df = df.withColumn(c, col(c).cast("double"))

df = df.dropna(subset=numeric_columns)
```

Cell 2 — Create features

```
from pyspark.ml.feature import VectorAssembler

feature_columns = [
    "monoisotopic_mass",
    "log1p_substances",
    "log1p_pubmed",
    "log1p_patents",
    "pop",
    "substances_approx",
    "pubmed_approx",
    "patents_approx",
    "np_lotus",
    "np_npatlas"
]

assembler = VectorAssembler(
    inputCols=feature_columns,
    outputCol="features"
)

data = assembler.transform(df).select(
    "features",
    col("np_coconut").alias("label")
)

data.show(5, truncate=False)
```

Cell 3 — Decision Tree Regression

This is the main **separate Decision Tree cell**:

```
from pyspark.ml.regression import DecisionTreeRegressor
from pyspark.ml.evaluation import RegressionEvaluator

# Train-test split
train_data, test_data = data.randomSplit([0.8, 0.2], seed=42)

# Create Decision Tree
dt = DecisionTreeRegressor(
    featuresCol="features",
    labelCol="label",
    maxDepth=5
)

# Train model
dt_model = dt.fit(train_data)

# Prediction
predictions = dt_model.transform(test_data)

predictions.select(
    "label",
    "prediction"
).show(10)
```

Cell 4 — Evaluate the Decision Tree

```
evaluator_rmse = RegressionEvaluator(
    labelCol="label",
    predictionCol="prediction",
    metricName="rmse"
)

evaluator_mae = RegressionEvaluator(
    labelCol="label",
    predictionCol="prediction",
    metricName="mae"
)

evaluator_r2 = RegressionEvaluator(
    labelCol="label",
    predictionCol="prediction",
    metricName="r2"
)

rmse = evaluator_rmse.evaluate(predictions)
```

```
mae = evaluator_mae.evaluate(predictions)
r2 = evaluator_r2.evaluate(predictions)

print("Decision Tree Regression")
print("------------------------")
print("RMSE :", rmse)
print("MAE  :", mae)
print("R²   :", r2)
```

Cell 5 — View the Decision Tree

One useful advantage of a Decision Tree is that you can inspect its rules:

```
print(dt_model.toDebugString)
```

You will get output similar to:

```
DecisionTreeRegressionModel
If (feature 1 <= ...)
   If (feature 5 <= ...)
      Predict ...
   Else
      Predict ...
Else
   Predict ...
```

Compare with your Linear Regression

Your Linear Regression results were:

Model	RMSE	MAE	R²
Linear Regression	0.4003	0.3519	0.2947
Decision Tree	Run model	Run model	Run model

Interpretation: For your dataset, the better model would generally have **lower RMSE/MAE and higher R²**.

One important point: since `np_coconut` is being predicted from `np_lotus` and `np_npatlas` along with other variables, check whether those database-count columns are conceptually available at prediction time. If they are derived from the same underlying information as `np_coconut`, they could introduce **data leakage or strong feature overlap**.